

API Testing Documentation

Project: Fake Store API Testing

1. Objective

The objective of this project is to test the APIs provided by the Fake Store API to validate their functionality, reliability, and response accuracy using API testing techniques.

The testing ensures that:

- APIs return correct responses for valid requests
- APIs handle invalid inputs properly
- Server communication works as expected
- Proper HTTP status codes and messages are returned
- CRUD operations behave correctly

2. Application Under Test

Fake Store API

Fake Store API is a public REST API used for practicing API testing.

It provides endpoints for:-

- Products
- Carts
- Users
- Authentication

3. Tools Used

Tool	Purpose
Postman	Sending API requests and validating responses
Browser	Accessing API documentation

4. Scope of Testing

The following APIs were tested:

- Product Module
- Cart Module
- User Module
- Authentication Module

5. Types of Testing Performed

- Functional Testing
- Positive Testing
- Negative Testing
- CRUD Operation Testing
- Status Code Validation
- Response Validation
- Data Integrity Testing

6. HTTP Methods Used

Method	Purpose
GET	Retrieve data
POST	Create or submit data
PUT	Update existing data
DELETE	Remove data

7. API Endpoints Tested

API Name	Method	Endpoint
Get all products	GET	/products
Get product by ID	GET	/products/{id}
Add product	POST	/products
Update product	PUT	/products/{id}
Delete product	DELETE	/products/{id}
Get all carts	GET	/carts
Get cart by ID	GET	/carts/{id}
Get user carts	GET	/carts/user/{userId}
Add cart	POST	/carts
Update cart	PUT	/carts/{id}
Delete cart	DELETE	/carts/{id}
Get all users	GET	/users
Get user by ID	GET	/users/{id}
Add user	POST	/users
Update user	PUT	/users/{id}
Delete user	DELETE	/users/{id}

8. API Testing Workflow

1. Understand API documentation
2. Identify endpoints and methods
3. Prepare request in Postman
4. Send API request
5. Validate:
 - Status code
 - Response body

- Data accuracy
6. Perform negative testing
 7. Document results

9. Sample API Test Cases

9.1 Products Module

TC_ID	Test Case	Method	Expected Result	Status
TC_P1	Get all products	GET	List of products returned	Pass
TC_P2	Get product by ID	GET	Product details returned	Pass
TC_P3	Add product	POST	Product created	Pass
TC_P4	Update product	PUT	Product updated	Pass
TC_P5	Delete product	DELETE	200 OK (mock delete behavior)	Pass

9.2 Cart Module

TC_ID	Test Case	Method	Expected Result	Status
TC_C1	Get all carts	GET	List of carts returned	Pass
TC_C2	Get cart by ID	GET	Cart details returned	Pass
TC_C3	Get user carts	GET	User cart data returned	Pass
TC_C4	Add cart	POST	Cart created	Pass
TC_C5	Update cart	PUT	Cart updated	Pass
TC_C6	Delete cart	DELETE	200 OK response (mock behavior)	Pass

9.3 User Module

TC_ID	Test Case	Method	Expected Result	Status
TC_U1	Get all users	GET	Users list returned	Pass
TC_U2	Get user by ID	GET	User details returned	Pass
TC_U3	Add user	POST	User created	Pass
TC_U4	Update user	PUT	User updated	Pass
TC_U5	Delete user	DELETE	200 OK response (mock behavior)	Pass

9.4 Authentication Module

TC_ID	Test Case	Method	Expected Result	Status
TC_A1	Login valid credentials	POST	Token generated	Pass
TC_A2	Login invalid credentials	POST	Error message	Pass

10. Sample API Request & Response

Request:-

GET <https://fakestoreapi.com/products>

Response:-

```
[  
  
  {  
  
    "id": 1,  
  
    "title": "Fjallraven Backpack",
```

```
"price": 109.95,  
  
"category": "men's clothing"  
  
}  
  
]
```

11. Validation Points

During testing, the following validations were performed:-

- HTTP status code validation
- Response structure validation
- Data type validation
- CRUD operation validation
- Error handling validation

12. Common Status Codes Observed

Status Code	Description
200	Success
201	Created successfully
400	Bad Request
404	Not Found
405	Method not allowed
500	Server error

13. Challenges Faced

- DELETE API returns 200 OK
- Data is not permanently removed
- GET request still returns same data

14. Learning Outcomes

After completing this task, the following skills were gained:

- Understanding REST API concepts
- Performing CRUD operations using Postman
- Writing API test cases
- Validating API responses
- Identifying real vs mock API behavior
- Writing professional QA documentation

15. Conclusion

API testing was successfully performed on the Fake Store API using Postman. All modules including Products, Carts, Users, and Authentication were validated.

The APIs responded correctly for GET, POST, and PUT operations. However, DELETE operations behave as simulated actions and do not persist changes, which is expected behavior in this demo API.